

PROPOSAL TUGAS AKHIR

**EVALUASI GITOPS MENGGUNAKAN ARGOCD TERHADAP
CONFIGURATION DRIFT PADA DEPLOYMENT APLIKASI
MULTI-SERVICE DI KLASSTER KUBERNETES: STUDI KASUS
INVENIORDM**

***EVALUATING GITOPS WITH ARGOCD FOR REDUCING
CONFIGURATION DRIFT IN MULTI-SERVICE APPLICATION
DEPLOYMENTS ON KUBERNETES CLUSTERS: A CASE STUDY OF
INVENIORDM***



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

PROPOSAL TUGAS AKHIR

EVALUASI GITOPS MENGGUNAKAN ARGOCD TERHADAP CONFIGURATION DRIFT PADA DEPLOYMENT APLIKASI MULTI-SERVICE DI KLASER KUBERNETES: STUDI KASUS INVENIORDM

EVALUATING GITOPS WITH ARGOCD FOR REDUCING CONFIGURATION DRIFT IN MULTI-SERVICE APPLICATION DEPLOYMENTS ON KUBERNETES CLUSTERS: A CASE STUDY OF INVENIORDM

Usulan Penelitian



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

HALAMAN PENGESAHAN

PROPOSAL TUGAS AKHIR

EVALUASI GITOPS MENGGUNAKAN ARGOCd TERHADAP CONFIGURATION DRIFT PADA DEPLOYMENT APLIKASI MULTI-SERVICE DI KLASER KUBERNETES: STUDI KASUS INVENIORDM

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 3 Juni 2026

Susunan Tim Penguji

Guntur Budi Herwanto, S.Kom., M.Cs.
Pembimbing

Nama Dosen Penguji 1
Ketua Penguji

Nama Dosen Penguji 2
Anggota Penguji

DAFTAR ISI

Halaman Judul	ii
Halaman Pengesahan	iii
INTISARI	vii
ABSTRACT	viii
I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	4
1.5 Hipotesis Penelitian	4
1.5.1 Hipotesis Nol (H_0)	5
1.5.2 Hipotesis Alternatif (H_1)	5
1.6 Manfaat Penelitian	5
1.7 Sistematika Penulisan	5
II PENELITIAN TERKAIT	7
2.1 Configuration Drift pada Lingkungan Kubernetes	7
2.1.1 Faktor yang Menimbulkan Configuration Drift	7
2.2 Dasar Teori: Rekonsiliasi dan Konvergensi pada Sistem Deklaratif . .	8
2.2.1 Konvergensi Menuju Desired State	8
2.2.2 Audit Trail sebagai Kontrol Sosial	9
2.3 Evaluasi Empiris GitOps dan ArgoCD	9
2.4 Metode Eksperimental Terkontrol pada Riset Sistem	10
2.5 InvenioRDM sebagai Beban Uji (Workload)	11
2.6 Kesenjangan Penelitian (<i>Research Gap</i>)	12
III METODE DAN RANCANGAN PENELITIAN	14
3.1 Deskripsi Umum Penelitian	14
3.2 Metodologi	15
3.3 Definisi Variabel Penelitian	17

3.3.1	Variabel Independen (Variabel Bebas)	17
3.3.2	Variabel Dependen (Variabel Terikat)	18
3.4	Lingkungan Eksperimen	20
3.4.1	Klaster Kubernetes	20
3.4.2	ArgoCD (Lingkungan B)	20
3.4.3	Beban Uji (Workload)	20
3.4.4	Kontrol Variabel Pengganggu	21
3.4.5	Protokol Pemantauan Variabel Terkontrol	21
3.5	Analisis Power dan Justifikasi Ukuran Sampel	21
3.6	Matriks Eksperimen	22
3.7	Skenario Drift Terkontrol	22
3.7.1	Skenario A — Perubahan Replika (Replica Drift)	23
3.7.2	Skenario B — Perubahan Image (Image Drift)	23
3.7.3	Skenario C — Perubahan ConfigMap (ConfigMap Drift)	23
3.7.4	Skenario D — Penghapusan Sumber Daya (Resource Deletion)	23
3.7.5	Skenario E — Patch Manual Tidak Sah (Unauthorized Manual Patch)	23
3.7.6	Skenario F — Drift Lintas Namespace (Cross-namespace Drift)	23
3.7.7	Skenario G — Perubahan Secret (Secret Drift)	24
3.8	Protokol Pengukuran	24
3.8.1	Protokol Pengukuran Baseline (O1)	24
3.8.2	Protokol Injeksi Drift (X)	24
3.8.3	Protokol Pengukuran Pasca-Injeksi (O2)	24
3.9	Metode Pengujian (Eksperimen E1 s.d. E4)	25
3.9.1	E1: Konsistensi Deployment (R1)	25
3.9.2	E2: Waktu Deteksi dan Pemulihan Drift (R2)	26
3.9.3	E3: Jumlah Intervensi Manual (R3)	26
3.9.4	E4: Jejak Audit Operasional (R4)	27
3.10	Analisis Statistik	27
3.11	Ancaman terhadap Validitas	28
3.11.1	Validitas Internal	28
3.11.2	Validitas Eksternal	28
3.11.3	Validitas Konstruk	29
3.11.4	Reliabilitas	29

IV JADWAL PENELITIAN	30
4.1 Penjelasan Kegiatan	30

INTISARI

Evaluasi GitOps menggunakan ArgoCD terhadap Configuration Drift pada Deployment Aplikasi Multi-Service di Kluster Kubernetes: Studi Kasus InvenioRDM

Oleh

Muhammad Argya Vityasy

23/522547/PA/22475

Deployment aplikasi multi-service pada kluster Kubernetes menggunakan pendekatan manual sering menimbulkan *configuration drift* akibat intervensi manusia, yang mengakibatkan ketidakkonsistenan *state* kluster, jejak audit yang buruk, dan waktu pemulihan operasional yang lama. Penelitian ini mengevaluasi secara empiris apakah manajemen deployment berbasis GitOps menggunakan ArgoCD dapat mengurangi *configuration drift* dan meningkatkan keandalan operasional dibandingkan dengan alur kerja deployment manual konvensional menggunakan `kubectl`. Penelitian mengadopsi desain eksperimental terkontrol dengan dua lingkungan paralel: Lingkungan A (deployment manual) dan Lingkungan B (deployment GitOps). Pada kedua lingkungan, *configuration drift* disuntikkan secara terkontrol melalui tujuh skenario (perubahan replika, perubahan *image*, modifikasi ConfigMap, penghapusan sumber daya, *patch* manual, *drift* lintas *namespace*, dan perubahan Secret). Pengukuran dilakukan terhadap empat metrik utama: (1) konsistensi deployment; (2) waktu deteksi dan pemulihan *drift*; (3) jumlah intervensi manual pasca-drift; dan (4) tingkat jejak audit (*traceability*). Setiap skenario diulang minimal lima kali per lingkungan untuk menghasilkan data yang dapat diuji secara statistik. Hipotesis nol (H0) menyatakan bahwa tidak terdapat perbedaan signifikan antara kedua pendekatan, sedangkan hipotesis alternatif (H1) memprediksi bahwa GitOps secara signifikan meningkatkan konsistensi deployment dan mengurangi waktu pemulihan drift. Hasil penelitian ini diharapkan memberikan bukti empiris mengenai dampak operasional GitOps dalam mengurangi *configuration drift* pada lingkungan Kubernetes.

ABSTRACT

Evaluating GitOps with ArgoCD for Reducing Configuration Drift in Multi-Service Application Deployments on Kubernetes Clusters: A Case Study of InvenioRDM

By

Muhammad Argya Vityasy

23/522547/PA/22475

Manual deployment of multi-service applications on Kubernetes clusters frequently introduces configuration drift due to human intervention, resulting in inconsistent cluster state, poor audit trails, and lengthy operational recovery time. This research empirically evaluates whether GitOps-based deployment management using ArgoCD can reduce configuration drift and improve operational reliability compared to conventional manual deployment workflows using kubectl. The study adopts a controlled experimental design with two parallel environments: Environment A (manual deployment) and Environment B (GitOps deployment). On both environments, controlled configuration drift is injected through seven scenarios (replica change, image change, ConfigMap modification, resource deletion, manual patch, cross-namespaces drift, and Secret change). Measurements are taken across four primary metrics: (1) deployment consistency; (2) drift detection and recovery time; (3) manual intervention count post-drift; and (4) operational traceability. Each scenario is repeated at least five times per environment to yield statistically testable data. The null hypothesis (H0) states that there is no significant difference between the two approaches, while the alternative hypothesis (H1) predicts that GitOps significantly improves deployment consistency and reduces drift recovery time. The results of this study are expected to provide empirical evidence on the operational impact of GitOps in reducing configuration drift in Kubernetes environments.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kubernetes telah menjadi platform orkestrasi kontainer yang dominan di industri maupun akademisi, digunakan oleh lebih dari 5,6 juta developer secara global [Cloud Native Computing Foundation, 2024]. Kemampuan Kubernetes dalam mengelola deployment, *scaling*, dan *self-healing* menjadikannya pilihan utama untuk menjalankan aplikasi yang memerlukan *availability* tinggi [Burns et al., 2016]. Namun, seiring bertambahnya jumlah layanan yang di-deploy pada sebuah klaster, kompleksitas operasional meningkat secara drastis. Sebuah aplikasi sekompleks InvenioRDM — platform repositori data penelitian *open-source* yang dikembangkan oleh CERN — terdiri dari setidaknya 16 komponen yang saling bergantung: layanan *web*, *worker* asinkron, database relasional, mesin pencari, *cache*, penyimpanan objek, *ingress controller*, manajer sertifikat, enkripsi *secret*, tunnel jaringan, kebijakan keamanan, sistem pemantauan, dan pencadangan [CERN, 2024].

Pengelolaan deployment aplikasi multi-service menggunakan pendekatan manual — melalui perintah `kubectl apply`, penskalaan manual, atau *patching* langsung pada klaster — sering menghasilkan ketidakkonsistenan operasional. Praktik ini rentan terhadap *configuration drift*, yaitu situasi di mana *state* aktual klaster menyimpang dari *state* yang didefinisikan secara deklaratif [Zhang et al., 2026]. Drift dapat timbul akibat tiga penyebab utama: (1) *perubahan langsung pada klaster* yang tidak tercatat dalam sistem *version control*; (2) *prosedur deployment yang tidak konsisten* antar operator; dan (3) *kurangnya mekanisme deteksi otomatis* yang dapat mengidentifikasi deviasi sebelum menimbulkan insiden.

Penelitian empiris oleh Zhang et al. [2026] menunjukkan bahwa 71% dari 2.260 skrip Kubernetes yang dianalisis mengandung *misconfiguration* yang bermakna, di mana sebagian besar menyebabkan *downtime* atau degradasi layanan. Rahman et al. [2023] menambahkan bahwa *security misconfiguration* pada manifest Kubernetes mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai root dan tanpa *resource limits*. Studi-studi ini mengonfirmasi bahwa drift bukan sekadar masalah estetika konfigurasi, melainkan ancaman nyata terhadap keandalan dan keamanan layanan.

GitOps menawarkan disiplin yang berpotensi mengatasi masalah ini. Dengan menjadikan repositori Git sebagai *single source of truth*, seluruh *state* infrastruktur tercatat, terverifikasi, dan dapat diaudit [CNCF GitOps Working Group, 2021]. ArgoCD mengimplementasikan prinsip-prinsip GitOps melalui *reconciliation loop* yang berjalan di dalam kluster: agen secara periodik membandingkan *state* Git dengan *state* aktual, dan secara otomatis menyelaraskan keduanya (*negative feedback controller*) [Argo Project, 2024]. Secara teoretis, model ini menjamin *convergence* — sistem selalu bergerak menuju *desired state* yang dideklarasikan dalam Git [Limoncelli et al., 2018].

Beberapa penelitian mendukung klaim ini secara kualitatif. Nataraja [2025] menemukan bahwa pendekatan deklaratif (GitOps/ArgoCD) menghasilkan *drift correction* yang lebih cepat, *compliance rate* yang lebih tinggi, dan paparan kerentanan yang lebih rendah dibandingkan pendekatan imperatif. Shrestha and Ali [2024] mengevaluasi GitOps versus Ansible untuk manajemen konfigurasi Kubernetes dan melaporkan keunggulan GitOps pada aspek *drift detection* dan *remedy time*. Matubber [2025] mengukur *reconciliation time* dan *reliability* GitOps, termasuk *self-healing* terhadap *configuration drift*.

Namun, temuan-temuan ini mengandung kelemahan metodologis yang belum teratasi. Shrestha and Ali [2024] menggunakan penilaian kualitatif untuk tingkat keparahan drift alih-alih operasionalisasi kuantitatif yang dapat direplikasi. Nataraja [2025] tidak mengisolasi efek pendekatan deployment dari variabel pengganggu seperti ukuran kluster dan tingkat keahlian tim. Matubber [2025] mengukur waktu rekonsiliasi tanpa membandingkan secara terkontrol dengan alur kerja manual pada kondisi eksperimental yang setara. Tanpa eksperimen terkontrol yang memitigasi variabel pengganggu dan menerapkan uji hipotesis formal, klaim bahwa GitOps secara kausal mengurangi drift tidak dapat dibuktikan — efek yang diamati mungkin atribut dari faktor ekstraneous (keahlian operator, karakteristik *workload*, versi *tool*) dan bukan dari paradigma deployment itu sendiri.

Kesenjangan ini menjadi landasan penelitian ini, yang mengevaluasi secara empiris dampak GitOps menggunakan ArgoCD terhadap *configuration drift* pada sebuah *workload* multi-service (InvenioRDM) yang di-deploy pada kluster Kubernetes, melalui desain eksperimental terkontrol dengan empat metrik operasional yang terdefinisi secara eksplisit dan analisis statistik formal.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana tingkat konsistensi deployment pada lingkungan GitOps (ArgoCD) dibandingkan dengan lingkungan deployment manual (`kubectl`) setelah diberikan skenario *configuration drift* terkontrol?
2. Berapa lama waktu yang dibutuhkan untuk mendeteksi dan memulihkan *configuration drift* pada lingkungan GitOps dibandingkan dengan lingkungan manual?
3. Berapa banyak intervensi manual yang diperlukan untuk memulihkan *state* kluster setelah insiden drift pada masing-masing lingkungan?
4. Bagaimana tingkat jejak audit (*traceability*) dan auditabilitas deployment pada lingkungan GitOps dibandingkan dengan lingkungan manual?

1.3 Batasan Masalah

Agar penelitian ini tetap fokus dan terukur, batasan masalah yang ditetapkan adalah:

1. **Variabel Independen:** Pendekatan manajemen deployment dengan dua level — (a) deployment manual menggunakan `kubectl`, dan (b) deployment GitOps menggunakan ArgoCD.
2. **Variabel Dependen:** Empat metrik utama — (1) *konsistensi deployment* (persentase), (2) *waktu deteksi dan pemulihan drift* (detik), (3) *jumlah intervensi manual* (jumlah tindakan korektif), dan (4) *tingkat jejak audit / traceability* (skor rubrik 1–5, MTTI, serta kelengkapan audit log).
3. Penelitian dilakukan pada kluster Kubernetes tunggal yang dikelola Rancher dengan spesifikasi 3 node, 24 total inti CPU, dan ~23 Gi total memori; skenario *multi-cluster* tidak termasuk dalam ruang lingkup.

4. Studi kasus yang digunakan adalah deployment InvenioRDM beserta 16 komponen infrastruktur dan dependensinya. Komponen InvenioRDM sebagai aplikasi (fitur, antarmuka, logika bisnis) bukan variabel penelitian — InvenioRDM merupakan subjek uji, bukan objek penelitian.
5. Skenario *drift* yang diuji terbatas pada tujuh kategori: (A) perubahan replika, (B) perubahan *image*, (C) modifikasi ConfigMap, (D) penghapusan sumber daya, (E) *patch* manual, (F) *drift* lintas *namespace*, dan (G) perubahan Secret.
6. Analisis statistik menggunakan uji Shapiro-Wilk untuk normalitas, dilanjutkan dengan *independent samples t-test* (jika data normal) atau *Mann-Whitney U test* (jika data tidak normal), dengan tingkat signifikansi $\alpha = 0,05$ dan ukuran efek (Cohen's d atau Cliff's δ).

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Mengukur dan membandingkan tingkat konsistensi deployment pada lingkungan GitOps (ArgoCD) dan lingkungan manual (`kubectl`) setelah skenario *configuration drift*.
2. Mengukur dan membandingkan waktu deteksi dan pemulihan *configuration drift* pada kedua lingkungan menggunakan metrik kuantitatif.
3. Mengukur dan membandingkan jumlah intervensi manual yang diperlukan oleh operator untuk memulihkan *state* kluster setelah insiden drift pada masing-masing lingkungan.
4. Mengukur dan membandingkan tingkat jejak audit (*traceability*) pada lingkungan GitOps dan lingkungan manual menggunakan rubrik operasional dan metrik kuantitatif.

1.5 Hipotesis Penelitian

Penelitian ini menguji hipotesis berikut:

1.5.1 Hipotesis Nol (H_0)

Tidak terdapat perbedaan yang signifikan secara statistik antara pendekatan deployment GitOps (ArgoCD) dan pendekatan deployment manual (`kubectl`) pada metrik: konsistensi deployment, waktu pemulihan drift, jumlah intervensi manual, dan tingkat jejak audit.

1.5.2 Hipotesis Alternatif (H_1)

Pendekatan deployment GitOps (ArgoCD) secara signifikan meningkatkan konsistensi deployment, mengurangi waktu pemulihan drift, menurunkan jumlah intervensi manual, dan meningkatkan tingkat jejak audit (*traceability*) dibandingkan dengan pendekatan deployment manual (`kubectl`).

1.6 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat berikut:

1. **Teoritis:** Memberikan bukti empiris terukur mengenai dampak operasional GitOps/ArgoCD pada aspek konsistensi deployment, pemulihan drift, dan jejak audit — yang selama ini banyak didasarkan pada klaim vendor atau studi kasus deskriptif. Hasil ini melengkapi literatur akademis dengan data eksperimental yang dapat direproduksi dan diverifikasi, serta memperkuat atau menantang klaim kausal yang belum teruji secara terkontrol.
2. **Metodologis:** Menawarkan protokol eksperimental yang dapat direplikasi — termasuk skenario drift terstandarisasi, metrik operasional, dan prosedur analisis statistik — sehingga penelitian selanjutnya dapat mereplikasi, memperluas, atau mengkritik temuan ini pada konteks dan platform yang berbeda.
3. **Praktis:** Menyediakan wawasan operasional yang dapat membantu administrator kluster Kubernetes dalam memilih pendekatan deployment yang sesuai berdasarkan bukti kuantitatif, serta menyoroti skenario drift yang paling rentan pada masing-masing pendekatan.

1.7 Sistematika Penulisan

Sistematika penulisan proposal tugas akhir ini adalah sebagai berikut:

- **Bab I:** Pendahuluan — berisi latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, hipotesis, manfaat penelitian, dan sistematika penulisan.
- **Bab II:** Penelitian Terkait — membahas studi empiris terkait *configuration drift* pada Kubernetes, evaluasi GitOps, dasar teori rekonsiliasi dan konvergensi, metode eksperimental pada riset sistem, serta *research gap* yang menjadi landasan penelitian.
- **Bab III:** Metode dan Rancangan Penelitian — menjelaskan metodologi studi eksperimental terkontrol, definisi variabel, tujuh skenario drift, protokol pengukuran, analisis statistik, ancaman terhadap validitas, dan lingkungan eksperimen.
- **Bab IV:** Jadwal Penelitian — memuat Gantt Chart berbasis bulan untuk perencanaan waktu penelitian selama 8 bulan.

BAB II

PENELITIAN TERKAIT

2.1 Configuration Drift pada Lingkungan Kubernetes

Configuration drift adalah situasi di mana *state* aktual sebuah sistem menyimpang dari *state* yang didefinisikan secara deklaratif [Pohjola, 2025]. Dalam konteks Kubernetes, drift terjadi ketika operator—secara langsung atau tidak sengaja—mengubah manifest klaster tanpa memperbarui sumber kebenaran deklaratif, sehingga *state* aktual dan *desired state* menjadi tidak konsisten.

Penelitian empiris oleh Zhang et al. [2026] melakukan studi terhadap 719 defect konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori defect, di mana 71% di antaranya menyebabkan *downtime* atau degradasi layanan. Studi ini menegaskan bahwa *misconfiguration* merupakan risiko operasional utama pada Kubernetes. Rahman et al. [2023] menambahkan bahwa *security misconfiguration* pada manifest Kubernetes *open-source* mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai root, tanpa *drop capabilities*, dan tanpa *resource limits*. Penelitian menunjukkan bahwa drift bukan hanya masalah estetika konfigurasi, tetapi ancaman nyata terhadap keandalan dan keamanan layanan.

Pohjola [2025] memberikan kerangka konseptual untuk memahami sumber dan dampak drift pada sistem *Infrastructure-as-Code* (IaC). Penelitian ini mengidentifikasi tiga faktor pendorong utama drift: (1) *keahlian manusia* yang tidak konsisten; (2) *tekanan waktu* yang memicu perubahan cepat dan tidak didokumentasikan; dan (3) *absensi mekanisme deteksi otomatis* pada pipeline deployment. Meskipun penelitian ini tidak beroperasi pada Kubernetes secara eksklusif, prinsip-prinsip yang dikemukakan berlaku umum untuk platform orkestrasi berbasis konfigurasi deklaratif.

2.1.1 Faktor yang Menimbulkan Configuration Drift

Berdasarkan literatur, faktor-faktor utama yang memperburuk *configuration drift* pada Kubernetes meliputi:

1. **Perubahan langsung pada klaster:** Penggunaan `kubectl edit`, `kubectl patch`, atau `kubectl scale` secara langsung tanpa memperbarui repositori sumber kebenaran [Zhang et al., 2026].

2. **Prosedur deployment yang tidak terdokumentasi:** Setiap operator dapat memiliki urutan perintah yang berbeda, yang mengakibatkan *state* kluster tidak deterministik [Pohjola, 2025].
3. **Kurangnya mekanisme *rollback* yang terintegrasi:** Deployment manual seringkali tidak memiliki mekanisme *rollback* otomatis; operator harus mengingat konfigurasi sebelumnya atau mengandalkan *backup* yang mungkin tidak konsisten [Rahman et al., 2023].
4. **Absensi deteksi drift otomatis:** Tanpa pengecekan berkala antara *state* aktual dan *desired state*, drift dapat terakumulasi selama periode lama tanpa terdeteksi [Zhang et al., 2026].

2.2 Dasar Teori: Rekonsiliasi dan Konvergensi pada Sistem Deklaratif

Keunggulan GitOps atas pendekatan imperatif dapat dijelaskan melalui dua prinsip teoretis yang saling berkaitan.

2.2.1 Konvergensi Menuju Desired State

Dalam teori sistem, sistem deklaratif yang menyimpan *desired state* dan secara kontinu berupaya mencapainya dikatakan bersifat *convergent* [Limoncelli et al., 2018]. ArgoCD mengimplementasikan prinsip ini melalui *reconciliation loop*: sebuah *negative feedback controller* yang secara periodik (1) membaca *desired state* dari repositori Git, (2) membandingkannya dengan *state* aktual di kluster, dan (3) melakukan tindakan korektif jika terdapat deviasi [Argo Project, 2024]. Mekanisme ini menjamin bahwa setiap deviasi (*drift*) bersifat *transien* — sistem secara otomatis kembali ke *desired state* tanpa memerlukan intervensi manusia, sepanjang *self-heal* diaktifkan.

Sebaliknya, pendekatan manual/imperatif tidak memiliki *feedback loop* bawaan. Setiap deviasi bersifat *persisten* sampai operator secara manual mendeteksi dan mengoreksinya. Tanpa mekanisme otomatis, interval antara drift dan pemulihan bergantung sepenuhnya pada kewaspadaan dan ketersediaan operator, yang bervariasi secara tidak deterministik [Pohjola, 2025].

2.2.2 Audit Trail sebagai Kontrol Sosial

GitOps memanfaatkan Git sebagai *append-only log* yang merekam setiap perubahan konfigurasi beserta metadata (penulis, waktu, pesan commit). Prinsip keempat OpenGitOps — *continuous reconciliation* — bergantung pada kemampuan Git untuk merekonstruksi keputusan konfigurasi secara retroaktif [CNCF GitOps Working Group, 2021]. Hal ini tidak hanya meningkatkan *traceability*, tetapi juga berfungsi sebagai *social control mechanism*: setiap perubahan tunduk pada *code review*, sehingga mengurangi kemungkinan perubahan yang tidak terverifikasi masuk ke produksi.

2.3 Evaluasi Empiris GitOps dan ArgoCD

GitOps adalah paradigma untuk mengelola infrastruktur dan aplikasi yang menggunakan repositori Git sebagai *single source of truth*. CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) *state* sistem yang diinginkan tersimpan dan berversi dalam Git; (3) perubahan *state* ditarik (*pulled*) secara otomatis oleh agen software; dan (4) agen software secara kontinu mendeteksi dan mengoreksi deviasi dari *state* yang diinginkan [CNCF GitOps Working Group, 2021].

Keunggulan GitOps dibandingkan deployment imperatif tradisional meliputi: *audit trail* lengkap melalui Git history, penerapan *code review* pada perubahan infrastruktur, *rollback* yang sederhana melalui `git revert`, dan reproduibilitas konfigurasi di berbagai lingkungan [Yuen et al., 2021]. Dalam model *pull-based* GitOps, agen yang berjalan di dalam kluster secara periodik membandingkan *state* Git dengan *state* kluster dan menyelaraskan keduanya, menghilangkan kebutuhan akan akses kredensial kluster dari luar [Utami and Fatta, 2021].

Utami and Fatta [2021] melakukan analisis perbandingan model deployment deklaratif *pull-based* (GitOps dengan ArgoCD) versus *push-based* pada Kubernetes, dan menemukan bahwa model *pull-based* lebih andal karena agen berjalan di dalam kluster dan tidak memerlukan kredensial eksternal. Shrestha and Ali [2024] mengevaluasi GitOps versus Ansible untuk manajemen konfigurasi Kubernetes, khususnya pada aspek *drift detection* dan *remedy time*; namun, studi ini mengandalkan penilaian kualitatif untuk menilai tingkat keparahan drift alih-alih metrik kuantitatif yang dapat direplikasi. Nataraja [2025] melakukan analisis berpusat keamanan terhadap pendekatan deklaratif (GitOps/ArgoCD) dan imperatif (Jenkins/kubectl), menemukan

bahwa pendekatan deklaratif menghasilkan *drift correction* yang lebih cepat, *compliance rate* yang lebih tinggi, dan paparan kerentanan yang lebih rendah; namun, studi ini tidak mengisolasi efek pendekatan deployment dari variabel pengganggu seperti ukuran klaster dan keahlian tim.

Paavola [2021] membandingkan ArgoCD dan FluxCD untuk pengelolaan multi-application pada Kubernetes dan menyimpulkan bahwa ArgoCD lebih cocok untuk skenario multi-application karena dukungan UI dan integrasi yang lebih baik. D'Amore [2021] mengimplementasikan ArgoCD untuk *continuous deployment* pada lingkungan *hybrid cloud*, namun tidak membahas evaluasi metrik kuantitatif secara eksplisit. Matubber [2025] mengukur *reconciliation time*, *scalability*, dan *reliability* GitOps untuk infrastruktur K8s, termasuk *self-healing* terhadap *configuration drift*; namun, pengukuran dilakukan tanpa kelompok pembanding (*control group*) yang menjalankan alur kerja manual, sehingga tidak dapat disimpulkan apakah waktu rekonsiliasi yang terukur lebih unggul daripada pemulihan manual.

Tabel 2.1 merangkum kelemahan metodologis kunci dari penelitian-penelitian di atas.

Tabel 2.1: Perbandingan Kelemahan Metodologis Penelitian Terdahulu

Penelitian	Fokus	Kelemahan Metodologis
Shrestha and Ali [2024]	GitOps vs Ansible	Penilaian kualitatif untuk drift severity; tidak ada uji hipotesis statistik
Nataraja [2025]	Keamanan deklaratif vs imperatif	Tidak mengisolasi confounding variables; desain observasional
Matubber [2025]	Reconciliation time GitOps	Tanpa control group (alur manual); tidak membandingkan secara terkontrol
Paavola [2021]	ArgoCD vs FluxCD	Komparatif deskriptif; tanpa metrik kuantitatif formal
D'Amore [2021]	ArgoCD hybrid cloud	Studi implementasi; tanpa evaluasi metrik

2.4 Metode Eksperimental Terkontrol pada Riset Sistem

Evaluasi empiris pada riset sistem dan infrastruktur memerlukan desain eksperimental yang memitigasi ancaman terhadap validitas [Wohlin et al., 2012]. Limoncel-

Illi et al. [2018] menjelaskan bahwa penelitian sistem harus membangun lingkungan yang dapat direproduksi, mengendalikan variabel pengganggu, dan mendefinisikan metrik yang jelas sebelum menyimpulkan hubungan kausal.

Desain eksperimental terkontrol yang digunakan dalam penelitian ini adalah *between-subjects design with matched pairs*: kedua lingkungan (manual dan GitOps) menerima perlakuan identik (skenario drift yang sama), sehingga setiap pasangan pengamatan antara Lingkungan A dan Lingkungan B pada skenario yang sama membentuk pasangan alami (*matched by scenario*). Hal ini memungkinkan pengujian perbedaan antara kedua pendekatan dengan mengendalikan variasi yang berasal dari jenis skenario drift.

Analisis data menggunakan uji statistik parametrik (*independent samples t-test*) jika distribusi data normal, atau non-parametrik (*Mann–Whitney U test*) jika tidak [Wohlin et al., 2012]. Prinsip-prinsip rigor statistik yang diterapkan meliputi: (1) pengulangan (*replication*) untuk memastikan reliabilitas; (2) kontrol variabel pengganggu untuk menjaga validitas internal; (3) ukuran efek (*effect size*) untuk menilai signifikansi praktis di luar signifikansi statistik; dan (4) analisis *power* untuk memastikan ukuran sampel memadai.

2.5 InvenioRDM sebagai Beban Uji (Workload)

InvenioRDM adalah platform repositori data penelitian *open-source* yang dikembangkan oleh CERN dan komunitas internasional, dirancang untuk memenuhi prinsip FAIR (*Findable, Accessible, Interoperable, Reusable*) [CERN, 2024]. Arsitektur InvenioRDM bersifat *microservices* yang terdiri dari beberapa komponen: (1) *web application* berbasis Flask/Gunicorn, (2) *Celery workers* untuk pemrosesan asinkron, (3) PostgreSQL untuk penyimpanan data, (4) OpenSearch untuk *indexing* dan pencarian, (5) Redis sebagai *cache* dan *message broker*, serta (6) MinIO/S3 untuk penyimpanan objek [Przerwa and Rohr, 2025].

InvenioRDM dipilih sebagai beban uji karena memenuhi tiga kriteria kerja: (a) kompleksitas dependensi yang memerlukan urutan deployment — *web* bergantung pada *database*, *cache*, dan *search engine*; (b) keberadaan komponen infrastruktur yang luas — mencakup *ingress*, TLS, *secrets*, monitoring, *backup*, dan kebijakan keamanan; dan (c) ketersediaan sebagai platform *open-source* yang dapat direproduksi [Chelakis, 2022, Fernández et al., 2016]. Dalam konteks penelitian ini, InvenioRDM berfungsi sebagai subjek uji (*test subject*), bukan objek penelitian. Segala temuan dan

metrik berlaku untuk workload secara umum, tidak terbatas pada InvenioRDM.

2.6 Kesenjangan Penelitian (*Research Gap*)

Berdasarkan tinjauan pustaka di atas, terdapat empat kesenjangan yang saling berkaitan dan menjadi landasan penelitian ini:

1. **Kesenjangan Validitas Konstruk:** Penelitian sebelumnya menunjukkan bahwa GitOps mengurangi drift, namun temuan-temuan ini bergantung pada metrik yang tidak teroperasionalisasi secara kuantitatif. Shrestha and Ali [2024] menggunakan penilaian kualitatif untuk tingkat keparahan drift, sementara Nataraja [2025] mengukur *compliance rate* tanpa mendefinisikan protokol pengukuran yang dapat direplikasi. Tanpa operasonalisasi metrik yang tegas, klaim keunggulan GitOps tidak dapat diverifikasi secara independen.
2. **Kesenjangan Validitas Internal:** Studi-studi yang ada tidak mengisolasi efek paradigma deployment dari variabel pengganggu. Nataraja [2025] tidak mengendalikan ukuran klaster atau keahlian operator; Matubber [2025] mengukur waktu rekonsiliasi GitOps tanpa kelompok pembanding (alur kerja manual). Akibatnya, efek yang diamati tidak dapat diatribusikan secara kausal kepada GitOps — mungkin disebabkan oleh keahlian operator yang lebih tinggi, karakteristik *workload* yang lebih sederhana, atau kondisi klaster yang lebih stabil.
3. **Kesenjangan Metodologis:** Tidak ditemukan penelitian yang menerapkan desain eksperimental terkontrol dengan uji hipotesis formal (H_0/H_1), uji normalitas, ukuran efek, dan analisis *power* pada pembandingan GitOps versus manual. Absensi rigor statistik ini mengakibatkan temuan yang ada tidak dapat digeneralisasi dan rentan terhadap *Type I error* (menerima klaim positif yang sebenarnya palsu).
4. **Kesenjangan Kontekstual:** Evaluasi GitOps yang ada berfokus pada lingkungan berskala besar atau *cloud provider* komersial. Belum ada evaluasi pada konteks klaster berukuran kecil-sedang yang mengelola *workload* akademis seperti InvenioRDM, di mana sumber daya operasional terbatas dan satu insiden drift dapat mengancam integritas data penelitian.

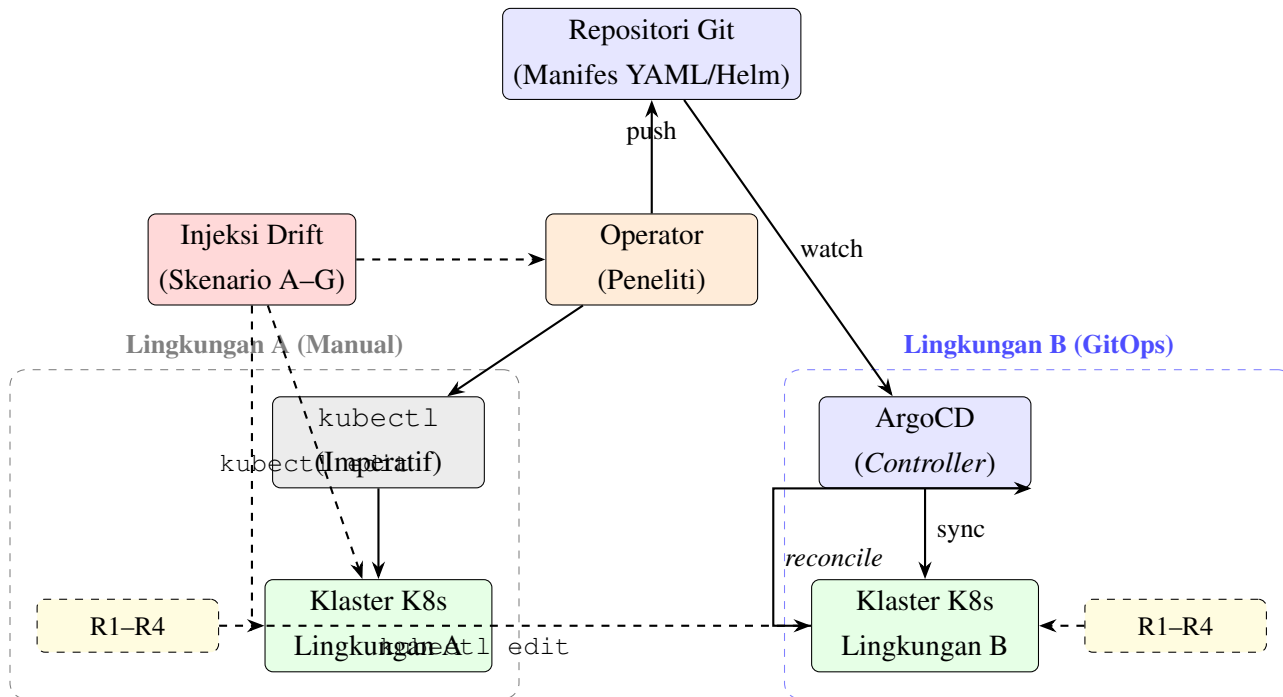
Penelitian ini mengisi keempat kesenjangan tersebut dengan: (1) mengoperasionalisasi empat metrik kuantitatif yang dapat direplikasi; (2) mengendalikan variabel pengganggu melalui desain eksperimental terkontrol (*matched-pairs between-subjects*); (3) menerapkan uji hipotesis formal dengan analisis statistik lengkap (normalitas, signifikansi, ukuran efek, analisis *power*); dan (4) mengevaluasi pada konteks klaster akademis dengan *workload* InvenioRDM.

BAB III

METODE DAN RANCANGAN PENELITIAN

3.1 Deskripsi Umum Penelitian

Penelitian ini mengevaluasi secara empiris dampak manajemen deployment berbasis GitOps menggunakan ArgoCD terhadap *configuration drift* pada sebuah kluster Kubernetes. Evaluasi dilakukan dengan membandingkan dua lingkungan paralel: Lingkungan A (deployment manual menggunakan `kubectl`) dan Lingkungan B (deployment GitOps menggunakan ArgoCD). Pada kedua lingkungan, *configuration drift* disuntikkan secara terkontrol melalui tujuh skenario yang dirancang untuk merepresentasikan kesalahan operasional yang umum terjadi. Hasil pengukuran terhadap empat metrik utama (konsistensi deployment, waktu pemulihan drift, jumlah intervensi manual, dan traceability) dianalisis menggunakan uji statistik untuk menguji hipotesis yang telah dirumuskan.



Gambar 3.1: Arsitektur eksperimen: perbandingan Lingkungan A (deployment manual dengan `kubectl`) dan Lingkungan B (deployment GitOps dengan ArgoCD). Injeksi drift dilakukan pada kedua lingkungan, dan metrik R1–R4 dikumpulkan dari masing-masing lingkungan.

3.2 Metodologi

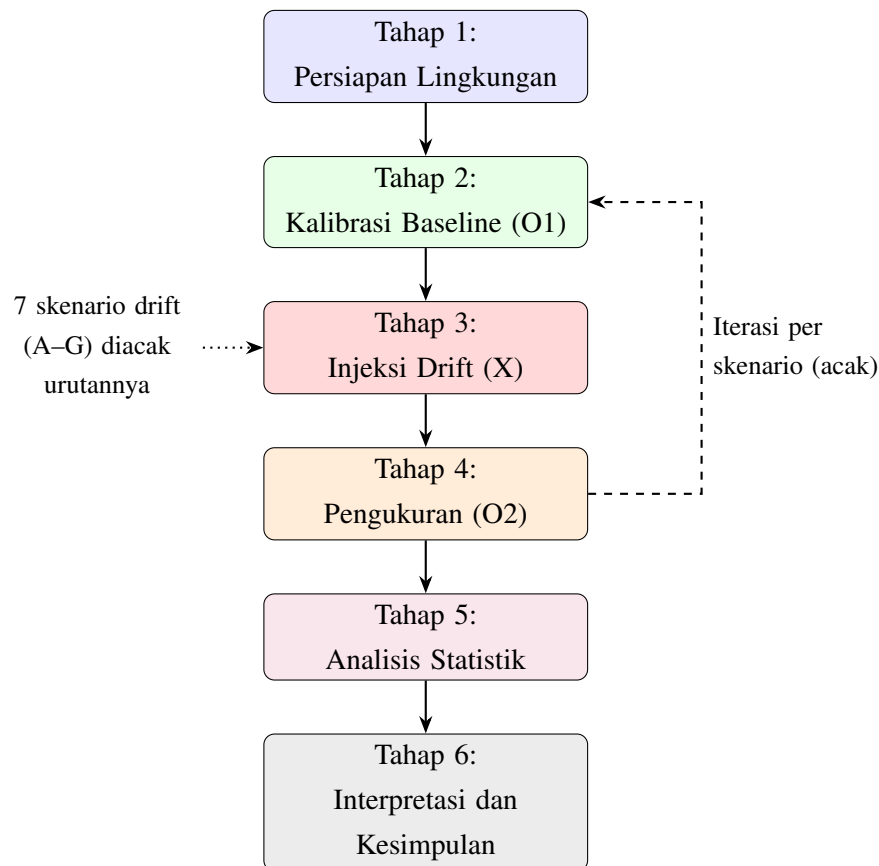
Metode penelitian yang digunakan adalah **Studi Eksperimental Terkontrol** dengan desain *between-subjects with matched pairs*. Dalam desain ini, kedua lingkungan (A dan B) menerima perlakuan identik (skenario drift yang sama), sehingga setiap pasangan pengamatan pada skenario tertentu membentuk pasangan alami (*matched by scenario*). Desain ini dipilih karena peneliti mengendalikan penuh kondisi eksperimen — baik penentuan lingkungan, urutan skenario, maupun injeksi drift — sehingga desain eksperimental (bukan kuasi-eksperimental) adalah klasifikasi yang tepat [Wohlin et al., 2012].

Tahapan eksperimen dalam penelitian ini mengikuti rancangan *pre-test – treatment – post-test*:

1. **Tahap 1: Persiapan Lingkungan** — Pembangunan Lingkungan A (manual) dan Lingkungan B (GitOps) pada klaster Kubernetes yang identik. Konfigurasi

klaster, versi ArgoCD, dan spesifikasi node disamakan untuk mengendalikan variabel pengganggu.

2. **Tahap 2: Kalibrasi Baseline (O1)** — Pengukuran *baseline* pada keadaan bersih sebelum injeksi drift. Mencakup verifikasi status deployment, pengukuran awal konsistensi, dan pencatatan metrik lingkungan (penggunaan CPU/memori, latensi jaringan).
3. **Tahap 3: Injeksi Drift (X)** — Penyuntikan *configuration drift* terkontrol pada kedua lingkungan menggunakan perintah identik (mis. `kubectl edit`, `kubectl scale`, `kubectl patch`). Urutan skenario diacak untuk menghindari efek urutan (*order effect*).
4. **Tahap 4: Pengukuran Pasca-Perlakuan (O2)** — Pengumpulan data metrik pasca-injeksi, termasuk waktu deteksi, waktu pemulihan, jumlah intervensi manual yang diperlukan, dan skor traceability.
5. **Tahap 5: Analisis Statistik** — Penerapan uji Shapiro–Wilk, *independent samples t-test* atau *Mann–Whitney U test*, serta perhitungan ukuran efek untuk menguji H0 dan H1.
6. **Tahap 6: Interpretasi dan Kesimpulan** — Pembahasan hasil statistik dalam konteks literatur dan praktik operasional.



Gambar 3.2: Bagan alir eksperimen: tahapan penelitian dari persiapan lingkungan hingga interpretasi dan kesimpulan. Iterasi per skenario menunjukkan pengulangan acak dari baseline hingga pengukuran untuk setiap skenario drift.

3.3 Definisi Variabel Penelitian

3.3.1 Variabel Independen (Variabel Bebas)

Variabel independen adalah pendekatan manajemen deployment dengan dua level:

1. **Lingkungan A — Deployment Manual:** Deployment dan modifikasi dilakukan langsung pada kluster menggunakan `kubectl apply`, `kubectl edit`, `kubectl scale`, dan perintah imperatif lainnya tanpa repositori Git sebagai sumber kebenaran.
2. **Lingkungan B — Deployment GitOps (ArgoCD):** Deployment dikelola secara deklaratif melalui repositori Git; ArgoCD berjalan di dalam kluster sebagai

agen rekonsiliasi yang secara kontinu menyelaraskan *state* klaster dengan *state* di Git.

3.3.2 Variabel Dependen (Variabel Terikat)

Empat metrik utama diukur sebagai variabel dependen:

Metrik R1: Konsistensi Deployment **Definisi:** Persentase sumber daya klaster (*Pod*, *Deployment*, *ConfigMap*, *Service*) yang cocok dengan *desired declarative state* setelah skenario drift.

Cara Ukur:

$$\text{Konsistensi (\%)} = \frac{\text{Jumlah sumber daya dengan status Healthy/Synced}}{\text{Total sumber daya yang diharapkan}} \times 100$$

Untuk Lingkungan B, status diambil dari ArgoCD Application status. Untuk Lingkungan A, status diverifikasi menggunakan `kubectl get` dan `kubectl describe`. Konsistensi diukur pada t_0 (sebelum drift), t_1 (sebelum pemulihan), dan t_2 (setelah pemulihan).

Metrik R2: Waktu Deteksi dan Pemulihan Drift **Definisi:** Waktu (dalam detik) mulai dari injeksi drift hingga drift terdeteksi dan dikoreksi.

- **Waktu Deteksi ($t_d - t_0$):** Untuk Lingkungan B, waktu dari injeksi drift hingga ArgoCD menunjukkan status *OutOfSync* pada aplikasi yang terkena drift (dipantau menggunakan `argocd app get` dan log ArgoCD). Untuk Lingkungan A, waktu hingga operator — tanpa prompt otomatis — secara manual mendeteksi deviasi melalui `kubectl get` atau monitoring.
- **Waktu Pemulihan ($t_2 - t_0$):** Untuk Lingkungan B, waktu dari injeksi drift hingga ArgoCD menyelesaikan rekonsiliasi otomatis (*self-heal*) dan menunjukkan status *Synced/Healthy*. Untuk Lingkungan A, waktu dari injeksi drift hingga operator menyelesaikan seluruh perintah `kubectl` manual yang diperlukan untuk mengembalikan *state* ke baseline.

Pengulangan: Setiap skenario diulang $n = 10$ kali per lingkungan untuk menghasilkan distribusi waktu yang memadai untuk analisis statistik.

Metrik R3: Jumlah Intervensi Manual **Definisi:** Jumlah tindakan korektif manual yang diperlukan untuk memulihkan *state* klaster setelah insiden drift.

Cara Ukur: Setiap perintah `kubectl` (`edit`, `patch`, `scale`, `rollout restart`, `delete`, `apply`) yang dieksekusi oleh operator pada Lingkungan A dihitung sebagai satu intervensi. Pada Lingkungan B, catat jumlah intervensi manual tambahan yang diperlukan ketika *self-heal* gagal atau tidak berjalan otomatis.

Perihal Lingkungan B: Untuk eksperimen R3, *self-heal* dinonaktifkan sementara agar operator harus memulihkan secara manual; hal ini memungkinkan perbandingan jumlah intervensi yang setara. Eksperimen terpisah dilakukan dengan *self-heal* aktif untuk mengukur *fully automated recovery*.

Metrik R4: Jejak Audit Operasional (Traceability) **Definisi:** Kemampuan untuk merekonstruksi siapa yang mengubah *state*, apa yang berubah, kapan perubahan terjadi, dan bagaimana pemulihan dilakukan.

Cara Ukur (Rubric 1–5):

1. **Skor 1:** Tidak ada jejak yang dapat direkonstruksi.
2. **Skor 2:** Hanya sebagian kecil jejak tersedia; mis. terdeteksi bahwa drift pernah terjadi, namun tidak jelas *who/what/when/how*.
3. **Skor 3:** Dua atau lebih aspek dapat diidentifikasi (mis. *what* dan *when*), namun satu atau lebih aspek tidak dapat dipastikan.
4. **Skor 4:** Tiga aspek dapat diidentifikasi dengan tingkat keyakinan tinggi.
5. **Skor 5:** Seluruh aspek (*who*, *what*, *when*, *how*) dapat direkonstruksi dengan bukti kuat dari log atau Git history.

Keandalan Penilai: Rubrik diisi oleh dua penilai independen; kesepakatan antar penilai diukur menggunakan *Cohen's kappa* (κ). Jika $\kappa < 0,70$, penilai melakukan diskusi untuk menyamakan persepsi dan mengisi ulang.

MTTI (Mean Time To Identify Drift): Waktu rata-rata dari injeksi drift hingga drift teridentifikasi dan terdokumentasikan. Lingkungan B: didasarkan pada notifikasi ArgoCD atau log *controller*. Lingkungan A: didasarkan pada Kubernetes audit logs dan waktu `kubectl get` pertama yang mengungkapkan drift.

Kelengkapan Audit Log: Persentase perubahan yang dapat direkonstruksi dari Git *commit history* (Lingkungan B) dibandingkan dengan Kubernetes audit logs (Lingkungan A).

3.4 Lingkungan Eksperimen

3.4.1 Klaster Kubernetes

Penelitian menggunakan klaster Kubernetes yang dikelola Rancher dengan spesifikasi hardware sebagai berikut:

- **Jumlah Node:** 3 node
- **Total CPU:** 24 inti
- **Total Memori:** ~23 Gi
- **Versi Kubernetes:** v1.29+
- **Container Runtime:** containerd
- **Sistem Operasi Node:** Ubuntu 22.04 LTS

3.4.2 ArgoCD (Lingkungan B)

- **Versi ArgoCD:** v2.12+
- **Konfigurasi Sync:** *Auto-sync* diaktifkan untuk eksperimen R2 (pemulihan otomatis), dimatikan untuk eksperimen R3 (intervensi manual). Interval sinkronisasi ditetapkan pada 60 detik untuk konsistensi waktu deteksi.
- **Konfigurasi Self-Heal:** Diaktifkan pada eksperimen R2; dinonaktifkan pada eksperimen R3.
- **Konfigurasi Prune:** *Pruning* diaktifkan agar sumber daya yang dihapus secara manual (Skenario D) dapat dideteksi dan diciptakan kembali oleh ArgoCD.

3.4.3 Beban Uji (Workload)

Deployment InvenioRDM dengan 16+ komponen: *web*, *worker*, PostgreSQL, OpenSearch, Redis, MinIO, Traefik, Cert-manager, Sealed Secrets, monitoring, *backup*, dan kebijakan keamanan. Komponen InvenioRDM sebagai aplikasi (fitur, antarmuka, logika bisnis) bukan variabel penelitian — InvenioRDM merupakan subjek uji, bukan objek penelitian.

3.4.4 Kontrol Variabel Pengganggu

Agar validitas internal tetap tinggi, variabel berikut dikendalikan secara sistematis:

1. **Interval Sinkronisasi ArgoCD:** Ditetapkan pada 60 detik (bukan default 180 detik) agar waktu deteksi konsisten.
2. **Tekanan Sumber Daya Node:** Rekam penggunaan CPU/memori kluster sebelum setiap pengujian. Hanya jalankan eksperimen jika node berada di bawah 70% kapasitas.
3. **Latensi Jaringan:** Gunakan kluster *on-premise* dengan latensi stabil. Catat RTT antar-node.
4. **Waktu Injeksi Drift:** Injeksi selalu dilakukan pada interval sinkronisasi ArgoCD untuk menghindari *edge case* waktu tunggu yang tidak dapat diprediksi.
5. **Versi ArgoCD:** Gunakan satu versi ArgoCD (v2.12+) sepanjang eksperimen.

3.4.5 Protokol Pemantauan Variabel Terkontrol

Variabel pengganggu dipantau secara kontinu selama eksperimen menggunakan skrip otomatis yang mencatat metrik sistem setiap 10 detik. Jika selama eksperimen terjadi pelanggaran batas (misalnya penggunaan CPU melebihi 70%), eksperimen dihentikan, data dari *run* tersebut dibuang, lingkungan dikalibrasi ulang ke baseline, dan *run* diulang. Seluruh kejadian pelanggaran dicatat dalam log eksperimen untuk transparansi.

3.5 Analisis Power dan Justifikasi Ukuran Sampel

Ukuran sampel ditentukan berdasarkan analisis *power a priori* menggunakan pendekatan berikut:

1. **Ukuran efek target:** Berdasarkan temuan Nataraja [2025] yang melaporkan perbedaan substansial antara pendekatan deklaratif dan imperatif, ukuran efek besar ($d = 1,0$, Cohen's convention) digunakan sebagai estimasi konservatif.
2. **Power dan signifikansi:** *Statistical power* ditetapkan pada $1 - \beta = 0,80$ dan $\alpha = 0,05$ (two-tailed).

3. **Ukuran sampel minimum (per grup):** Untuk *independent samples t-test* dengan $d = 1,0$, $\alpha = 0,05$, dan $power = 0,80$, ukuran sampel minimum per grup adalah $n = 17$ secara total per metrik [Cohen, 1988]. Namun, karena setiap skenario diulang secara independen ($7 \text{ skenario} \times n \text{ pengulangan per skenario}$), jumlah total observasi per metrik per lingkungan adalah $7 \times n$.
4. **Pemilihan n per skenario:** Untuk R2 (waktu pemulihan, metrik utama), dipilih $n = 10$ per skenario per lingkungan, menghasilkan $7 \times 10 = 70$ observasi per lingkungan — jauh melampaui minimum $n = 17$ secara total. Untuk R1, R3, dan R4, dipilih $n = 5$ per skenario per lingkungan, menghasilkan $7 \times 5 = 35$ observasi per lingkungan — masih memadai untuk mendeteksi efek besar. Jika hasil menunjukkan efek sedang ($d = 0,5$), pengulangan dapat ditambah pada tahap pengumpulan data tambahan.

3.6 Matriks Eksperimen

Tabel 3.1 menunjukkan rancangan eksperimental secara keseluruhan.

Tabel 3.1: Matriks Eksperimen

Dimensi	Lingkungan A (Manual)	Lingkungan B (GitOps)
Perlakuan	kubectl imperatif	ArgoCD deklaratif + Git
Skenario Drift	A, B, C, D, E, F, G	A, B, C, D, E, F, G
Pengulangan	$n = 5$ per skenario (R1, R3, R4); $n = 10$ per skenario (R2)	$n = 5$ per skenario (R1, R3, R4); $n = 10$ per skenario (R2)
Metrik	R1, R2, R3, R4	R1, R2, R3, R4
Baseline	kubectl get semua sum-ber daya sebelum drift	ArgoCD <i>App status</i> sebelum drift

3.7 Skenario Drift Terkontrol

Tujuh skenario drift dirancang untuk merepresentasikan kesalahan operasional yang umum. Setiap skenario dieksekusi secara identik pada kedua lingkungan. Untuk Lingkungan A, operator kemudian memulihkan secara manual. Untuk Lingkungan B, ArgoCD secara otomatis mendeteksi dan memulihkan (pada eksperimen R2 dengan *self-heal* aktif).

3.7.1 Skenario A — Perubahan Replika (Replica Drift)

Secara manual mengubah jumlah replika Deployment menggunakan `kubectl scale deployment/nama-deployment --replicas=5` ketika *desired* adalah 3. Perubahan ini melewati Git pada Lingkungan B.

3.7.2 Skenario B — Perubahan Image (Image Drift)

Secara manual mengganti versi image kontainer pada Deployment menggunakan `kubectl edit deployment/nama-deployment` dan mengubah kolom image ke versi yang berbeda.

3.7.3 Skenario C — Perubahan ConfigMap (ConfigMap Drift)

Mengubah isi ConfigMap langsung di dalam kluster menggunakan `kubectl edit configmap/nama-configmap`. Memodifikasi parameter konfigurasi seperti `DATABASE_URL` atau `LOG_LEVEL`.

3.7.4 Skenario D — Penghapusan Sumber Daya (Resource Deletion)

Menghapus Deployment atau Service secara manual menggunakan `kubectl delete deployment/nama-deployment`. Untuk Lingkungan B, ArgoCD dengan *prune* aktif akan mendeteksi sumber daya yang hilang dan menciptakan ulang dari manifest Git.

3.7.5 Skenario E — Patch Manual Tidak Sah (Unauthorized Manual Patch)

Menerapkan `kubectl patch` langsung terhadap sumber daya yang dikelola, mis. mengubah *resource limits* pada sebuah Pod menggunakan patch JSON.

3.7.6 Skenario F — Drift Lintas Namespace (Cross-namespace Drift)

Menerapkan sumber daya ke namespace yang salah secara manual. Mengukur apakah GitOps membatasi propagasi atau mendeteksi kesalahan namespace secara otomatis. Contoh: `kubectl apply -f deployment.yaml --namespace=wrong-names`

3.7.7 Skenario G — Perubahan Secret (Secret Drift)

Mengubah secret secara manual di luar Git. Skenario ini sangat umum dalam praktik produksi. Contoh: `kubectl patch secret my-secret --patch="{\"data\": {`

3.8 Protokol Pengukuran

3.8.1 Protokol Pengukuran Baseline (O1)

Sebelum setiap pengujian:

1. Verifikasi semua *pod* dalam status *Running* dan *healthy*.
2. Catat *timestamp* awal.
3. Catat penggunaan CPU/memori node (untuk kontrol variabel).
4. Simpan output `kubectl get all -n <namespace>` (Lingkungan A) atau `argocd app list` (Lingkungan B) sebagai baseline.
5. Verifikasi variabel pengganggu berada dalam batas (CPU < 70%, RTT stabil).

3.8.2 Protokol Injeksi Drift (X)

1. Pilih skenario secara acak dari daftar {A, B, C, D, E, F, G} untuk menghindari efek urutan.
2. Eksekusi perintah injeksi drift pada kedua lingkungan secara paralel atau berurutan dengan jeda tetap (5 menit).
3. Catat *timestamp* t_0 (waktu eksekusi perintah injeksi).

3.8.3 Protokol Pengukuran Pasca-Injeksi (O2)

1. **Untuk Lingkungan B (GitOps):** Pantau status ArgoCD menggunakan `argocd app get` setiap 5 detik. Catat: (a) waktu ArgoCD menunjukkan *OutOfSync* (deteksi); (b) waktu ArgoCD menunjukkan *Synced* setelah *auto-heal* (pemulihan). Hitung $t_d - t_0$ (waktu deteksi) dan $t_2 - t_0$ (waktu pemulihan total).

2. **Untuk Lingkungan A (Manual):** Operator melakukan pemantauan manual menggunakan `kubectl get` dan `kubectl describe`. Catat waktu eksekusi setiap perintah korektif. Hitung $t_2 - t_0$ (waktu total hingga *state* kembali ke baseline).
3. Catat jumlah perintah `kubectl` yang diperlukan (R3).
4. Dokumentasikan traceability: dua penilai independen mengisi rubrik skor 1–5 berdasarkan log yang tersedia (Git history untuk Lingkungan B; Kubernetes audit logs untuk Lingkungan A). Hitung Cohen’s kappa untuk kesepakatan antar penilai.

3.9 Metode Pengujian (Eksperimen E1 s.d. E4)

3.9.1 E1: Konsistensi Deployment (R1)

Tabel 3.2: Desain Eksperimen E1 — Konsistensi Deployment

Parameter	Deskripsi
Metrik	Persentase sumber daya yang cocok dengan <i>desired declarative state</i>
Cara Ukur	Hitung jumlah sumber daya Healthy/Synced dibagi total <i>expected resources</i>
Pendekatan	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD
Pengulangan	n = 5 per skenario per lingkungan

3.9.2 E2: Waktu Deteksi dan Pemulihan Drift (R2)

Tabel 3.3: Desain Eksperimen E2 — Waktu Deteksi dan Pemulihan Drift

Parameter	Deskripsi
Metrik	Waktu (detik) dari drift terjadi hingga terdeteksi (Lingkungan B) atau dipulihkan (kedua lingkungan)
Cara Ukur	Lingkungan B: pantau status ArgoCD setiap 5 detik; Lingkungan A: catat setiap perintah <code>kubectl</code> manual
Pendekatan	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD (dengan <i>self-heal</i> aktif)
Pengulangan	n = 10 per skenario per lingkungan

3.9.3 E3: Jumlah Intervensi Manual (R3)

Tabel 3.4: Desain Eksperimen E3 — Jumlah Intervensi Manual

Parameter	Deskripsi
Metrik	Jumlah tindakan korektif manual yang diperlukan
Cara Ukur	Hitung setiap perintah <code>kubectl</code> (edit, patch, scale, rollout restart, delete, apply)
Pendekatan	(a) Manual/ <code>kubectl</code> (tanpa bantuan otomatis), (b) GitOps/ArgoCD (dengan <i>self-heal</i> dimatikan)
Pengulangan	n = 5 per skenario per lingkungan

3.9.4 E4: Jejak Audit Operasional (R4)

Tabel 3.5: Desain Eksperimen E4 — Jejak Audit Operasional (Traceability)

Parameter	Deskripsi
Metrik	Skor rubrik 1–5 (<i>who/what/when/how</i>), MTTI (detik), kelengkapan audit log (%)
Cara Ukur	Lingkungan B: Git commit history + ArgoCD notifications; Lingkungan A: Kubernetes audit logs + <code>kubectl</code> timestamps; dua penilai independen
Pendekatan	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD
Pengulangan	n = 5 per skenario per lingkungan

3.10 Analisis Statistik

Setelah pengumpulan data selesai, analisis statistik dilakukan dengan langkah berikut:

1. **Uji Normalitas:** Terapkan *Shapiro–Wilk test* pada setiap distribusi metrik per lingkungan untuk menentukan apakah data berdistribusi normal.
2. **Uji Hipotesis:**
 - Jika data normal: gunakan *independent samples t-test* untuk membandingkan rata-rata metrik antara Lingkungan A dan Lingkungan B.
 - Jika data tidak normal: gunakan *Mann–Whitney U test* (non-parametrik).
3. **Tingkat Signifikansi:** Gunakan $\alpha = 0,05$. Jika $p < 0,05$, tolak H_0 dan terima H_1 .
4. **Ukuran Efek:**
 - Untuk *independent samples t-test*: hitung *Cohen’s d* untuk menilai signifikansi praktis.
 - Untuk *Mann–Whitney U test*: hitung *Cliff’s delta* untuk menilai signifikansi praktis.
5. **Interpretasi:** Jika perbedaan signifikan secara statistik ($p < 0,05$) dan ukuran efek besar ($|d| > 0,8$ atau $|\delta| > 0,474$), maka H_1 didukung kuat. Jika perbedaan tidak signifikan, H_0 gagal ditolak.

3.11 Ancaman terhadap Validitas

Berikut adalah identifikasi ancaman terhadap validitas beserta strategi mitigasi yang diterapkan, mengikuti kerangka Wohlin et al. [2012] dan Cook and Campbell [1979].

3.11.1 Validitas Internal

1. **Seleksi (*Selection Bias*):** Kedua lingkungan tidak ditugaskan secara acak dari populasi — peneliti membangun keduanya secara deliberatif. *Mitigasi:* Lingkungan A dan B dibangun pada kluster identik dengan konfigurasi yang disamakan; satu-satunya perbedaan adalah pendekatan deployment (IV).
2. **Maturasi (*Maturation*):** Kinerja operator dapat membaik sepanjang eksperimen karena pembelajaran. *Mitigasi:* Urutan skenario diacak; data pembelajaran operator dicatat; *practice run* dilakukan sebelum pengumpulan data resmi.
3. **Instrumentasi:** Perubahan pada alat ukur selama eksperimen (mis. pembaruan ArgoCD). *Mitigasi:* Seluruh versi software (ArgoCD, Kubernetes, InvenioRDM) dikunci sebelum eksperimen dimulai dan tidak diperbarui selama pengumpulan data.
4. **Efek Pengujian (*Testing*):** Operator mungkin berubah perilakunya karena sadar sedang diobservasi (*Hawthorne effect*). *Mitigasi:* Operator mengikuti *runbook* terstandarisasi; waktu reaksi — bukan kualitas keputusan — yang diukur pada R2.

3.11.2 Validitas Eksternal

1. **Kekhususan Konteks:** Hasil mungkin tidak generalisasi ke kluster berukuran besar, *workload* yang berbeda, atau tim operator yang lebih besar. *Mitigasi:* Dokumentasikan seluruh karakteristik kluster, *workload*, dan operator secara transparan; gunakan InvenioRDM yang mewakili kelas *workload* multi-service umum.
2. **Interaksi Seleksi dan Perlakuan:** Efek GitOps mungkin spesifik untuk tingkat keahlian operator dalam penelitian ini. *Mitigasi:* Catat profil keahlian operator; rekomendasikan replikasi dengan tim yang berbeda.

3.11.3 Validitas Konstruk

1. **Bias Operasi Tunggal (*Mono-operation Bias*):** Menggunakan satu *workload* (InvenioRDM) dan satu alat GitOps (ArgoCD). *Mitigasi:* InvenioRDM dipilih karena kompleksitas yang mewakili kelas *workload* multi-service; ArgoCD merupakan implementasi GitOps yang paling banyak digunakan.
2. **Bias Metode Tunggal (*Mono-method Bias*):** Metrik waktu dan konsistensi diukur secara otomatis, namun R4 menggunakan rubrik. *Mitigasi:* Dua penilai independen dengan Cohen’s kappa; MTTI dan kelengkapan audit log sebagai metrik kuantitatif pendamping rubrik.
3. **Operasionalisasi Konstruk:** Definisi “drift” dibatasi pada 7 skenario yang merepresentasikan kesalahan operasional umum. *Mitigasi:* Skenario dirancang berdasarkan taksonomi Zhang et al. [2026] dan praktik yang dilaporkan oleh Pohjola [2025]; batasan ini diakui secara eksplisit.

3.11.4 Reliabilitas

1. **Reliabilitas Pengukuran:** Pengukuran waktu (R2) bergantung pada presisi pencatatan *timestamp*. *Mitigasi:* Semua *timestamp* direkam secara otomatis oleh skrip pengukuran; akurasi diverifikasi dengan clock sinkronisasi (NTP).
2. **Reliabilitas Replikasi:** Pihak lain harus dapat mereplikasi eksperimen. *Mitigasi:* Protokol eksperimen didokumentasikan secara rinci; konfigurasi kluster, versi software, dan skrip drift injection akan dipublikasikan sebagai *artifact* pendamping.

BAB IV

JADWAL PENELITIAN

Penelitian ini direncanakan untuk dilaksanakan selama delapan bulan, dari bulan Juni 2026 hingga Januari 2027. Jadwal penelitian disusun berdasarkan tahapan studi eksperimental terkontrol: persiapan lingkungan, kalibrasi baseline, pelaksanaan eksperimen, analisis statistik, dan penulisan laporan.

Tabel 4.1 menunjukkan rencana jadwal pelaksanaan penelitian berbasis bulan. Simbol • menunjukkan bahwa kegiatan dilaksanakan pada bulan tersebut.

Tabel 4.1: Jadwal Penelitian (Gantt Chart)

Kegiatan	Bulan							
	Jun	Jul	Agu	Sep	Okt	Nov	Des	Jan
Studi Literatur	•	•						
Perancangan Lingkungan		•	•					
Implementasi ArgoCD + Baseline Manual			•	•				
Persiapan Baseline & Kalibrasi				•	•			
Eksperimen & Pengumpulan Data (E1–E4)					•	•		
Analisis Statistik & Interpretasi						•	•	
Penulisan Laporan	•	•	•	•	•	•	•	•
Seminar / Sidang								•

4.1 Penjelasan Kegiatan

1. Studi Literatur (Juni – Juli 2026)

Kegiatan ini mencakup pengumpulan dan analisis literatur mengenai: *configuration drift* pada Kubernetes, evaluasi empiris GitOps, metode eksperimental pada riset sistem, dan arsitektur InvenioRDM sebagai beban uji. Sumber literatur meliputi jurnal ilmiah (IEEE, ACM, Springer), dokumentasi resmi, buku teks, dan artikel teknis. Tujuan dari fase ini adalah membangun landasan teoretis dan menetapkan hipotesis yang dapat diuji.

2. Perancangan Lingkungan (Juli – Agustus 2026)

Tahap ini meliputi perancangan arsitektur Lingkungan A (deployment manual/`kubectl`) dan Lingkungan B (deployment GitOps/ArgoCD) pada kluster Kubernetes yang sama. Perancangan meliputi: definisi variabel independen dan dependen, pemilihan tujuh skenario drift, rancangan tabel rubrik traceability, dan spesifikasi protokol pengukuran.

3. Implementasi ArgoCD + Baseline Manual (Agustus – Oktober 2026)

Tahap implementasi meliputi: konfigurasi ArgoCD pada kluster, pembuatan repositori GitOps dengan manifest deklaratif, konfigurasi baseline manual (alur kerja `kubectl`), implementasi skenario drift (script injeksi otomatis), serta konfigurasi logging dan monitoring (Prometheus, Grafana, Kubernetes audit logs, ArgoCD notifications). Pada akhir fase ini, kedua lingkungan siap untuk eksperimen.

4. Persiapan Baseline & Kalibrasi (September – Oktober 2026)

Sebelum eksperimen dimulai, fase kalibrasi dilakukan untuk memastikan validitas internal. Kegiatan meliputi: (a) deployment InvenioRDM pada kedua lingkungan; (b) verifikasi baseline bersih; (c) pengukuran awal konsistensi; (d) verifikasi penggunaan sumber daya node di bawah 70%; dan (e) uji coba awal untuk memvalidasi protokol pengukuran.

5. Eksperimen dan Pengumpulan Data (Oktober – November 2026)

Pelaksanaan empat eksperimen sesuai desain pada Bab III. E1: konsistensi deployment ($n = 5$ per skenario). E2: waktu deteksi dan pemulihan drift ($n = 10$ per skenario). E3: jumlah intervensi manual ($n = 5$ per skenario). E4: traceability ($n = 5$ per skenario, lengkap dengan rubrik dan MTTI). Data dikumpulkan dalam bentuk *timestamp*, status aplikasi, dan log. Skenario diacak untuk setiap lingkungan untuk memitigasi *order effect*.

6. Analisis Statistik dan Interpretasi (November – Desember 2026)

Analisis data meliputi: uji normalitas (Shapiro–Wilk), uji hipotesis (*independent samples t-test* atau *Mann–Whitney U test*), perhitungan ukuran efek (Cohen's d atau Cliff's δ), dan interpretasi hasil dalam konteks H_0 dan H_1 . Pembahasan mencakup identifikasi skenario drift yang paling berdampak, faktor yang mempengaruhi hasil, serta ancaman terhadap validitas internal dan eksternal penelitian.

7. Penulisan Laporan (Juni 2026 – Januari 2027)

Penulisan laporan dilakukan secara paralel sepanjang penelitian. Setiap tahap menghasilkan bab atau bagian yang sesuai. Draft proposal diselesaikan pada pertengahan semester, draft skripsi lengkap pada akhir penelitian.

8. Seminar / Sidang (Januari 2027)

Presentasi dan sidang tugas akhir di depan dosen penguji, mencakup paparan hasil eksperimen, analisis statistik, dan kesimpulan.

DAFTAR PUSTAKA

- Argo Project. Argocd documentation, 2024. URL <https://argo-cd.readthedocs.io>.
- Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.
- CERN. Inveniordm documentation, 2024. URL <https://inveniordm.docs.cern.ch>.
- K. Chelakis. Creating an open archival information system compliant archive for cern. Master’s thesis, CERN, 2022.
- Cloud Native Computing Foundation. Cnfd cloud native definition v1.0, 2024. URL <https://www.cncf.io/about/who-we-are>.
- CNCF GitOps Working Group. Opengitops principles v1.0.0. <https://opengitops.dev>, 2021.
- Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.
- Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.
- M. D’Amore. Gitops and argocd: Continuous deployment and maintenance of a full stack application in a hybrid cloud kubernetes environment. Master’s thesis, Politecnico di Torino, 2021.
- J. D. Fernández, P. Fokianos, P. Herterich, and T. Šimko. Cern analysis preservation: A novel digital library service to enable reusable and reproducible research. In *Research and Advanced Technology for Digital Libraries*. Springer, 2016.
- Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.
- M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master’s thesis, KTH Royal Institute of Technology, 2025.

- P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master's thesis, National College of Ireland, 2025.
- E. Paavola. Managing multiple applications on kubernetes using gitops principles. Master's thesis, Turku University of Applied Sciences, 2021.
- O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master's thesis, Aalto University, 2025.
- K. Przerwa and S. Rohr. A modern open source integrated library system by invenio. In *International Conference on Theory and Practice of Digital Libraries (TPDL)*. Springer, 2025.
- A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.
- R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.
- Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.
- Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.
- Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.
- Y. Zhang, U. Paul, M. d'Amorim, and A. Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 2026.